BBPwned – User Manual

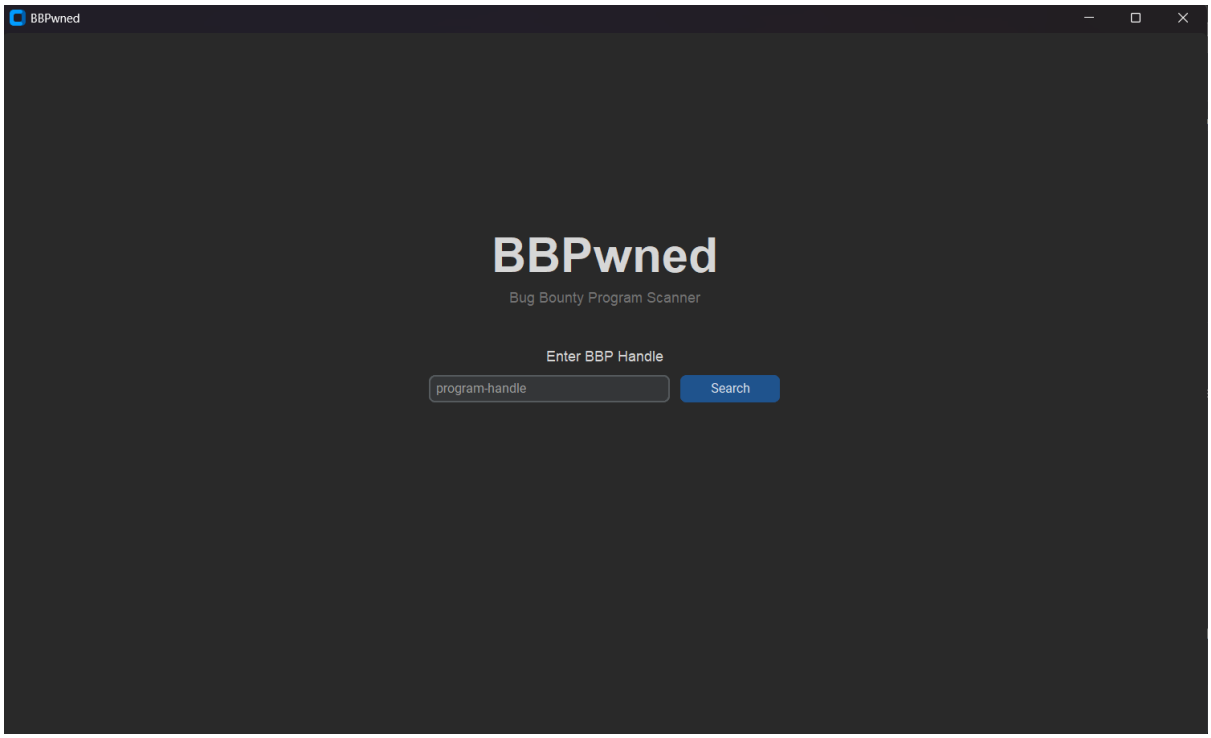
BBPwned is a HackerOne-based bug bounty hunting toolkit designed to assist less experienced bounty hunters. The toolkit includes automated and guided processes for all elements of a bug bounty hunt, from program selection to vulnerability testing and reporting.

Installation

BBPwned has been made available as a compiled executable. Just download the zip file, extract the executable and its dependencies and run the software.

Program Selection

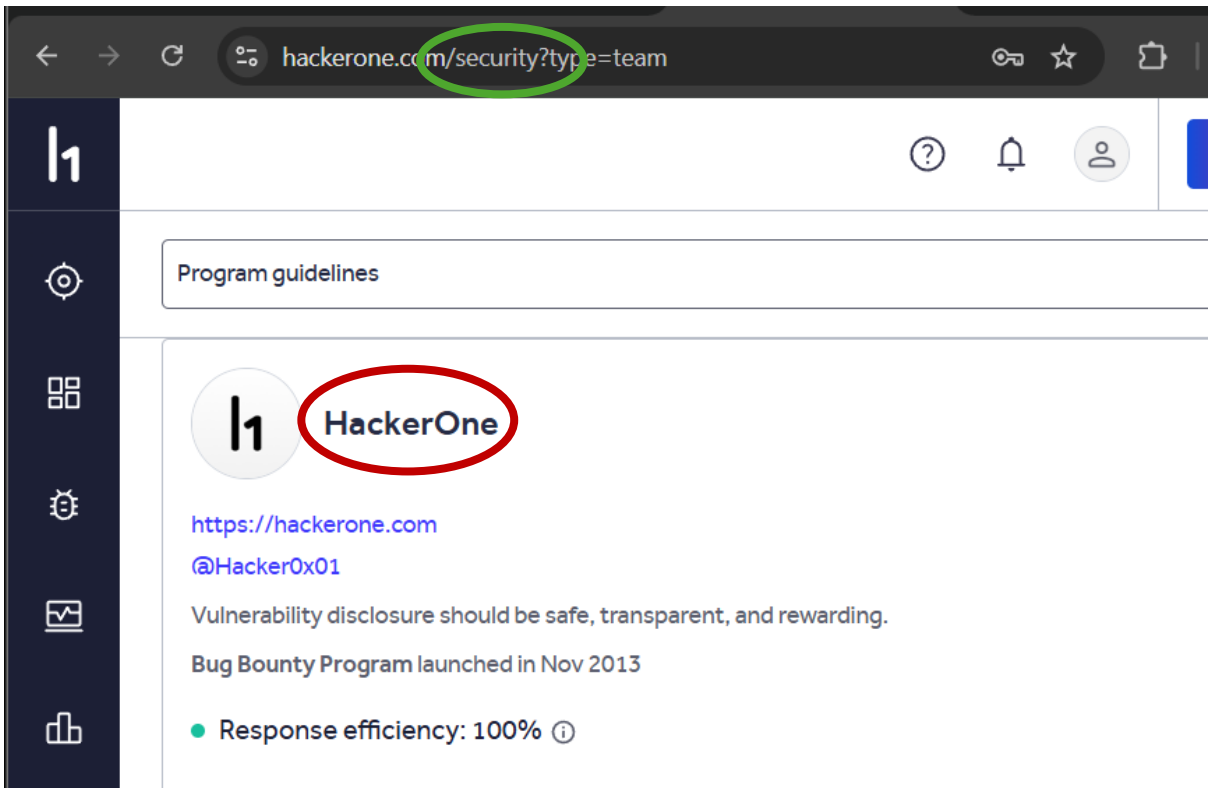
Upon opening the program, the following landing page should be seen:



This page is used to search for the desired program by its HackerOne program handle, simply type in an appropriate handle (usually the organisation's name) and the program info should be retrieved, starting the bug bounty hunt process. To restart the hunt and return to this page, click File -> New Project within the navbar.

NOTE: not all programs will have handles matching the organisation's name, if the search is unsuccessful – this is most likely the case. Luckily, finding out the program handle is very simple:

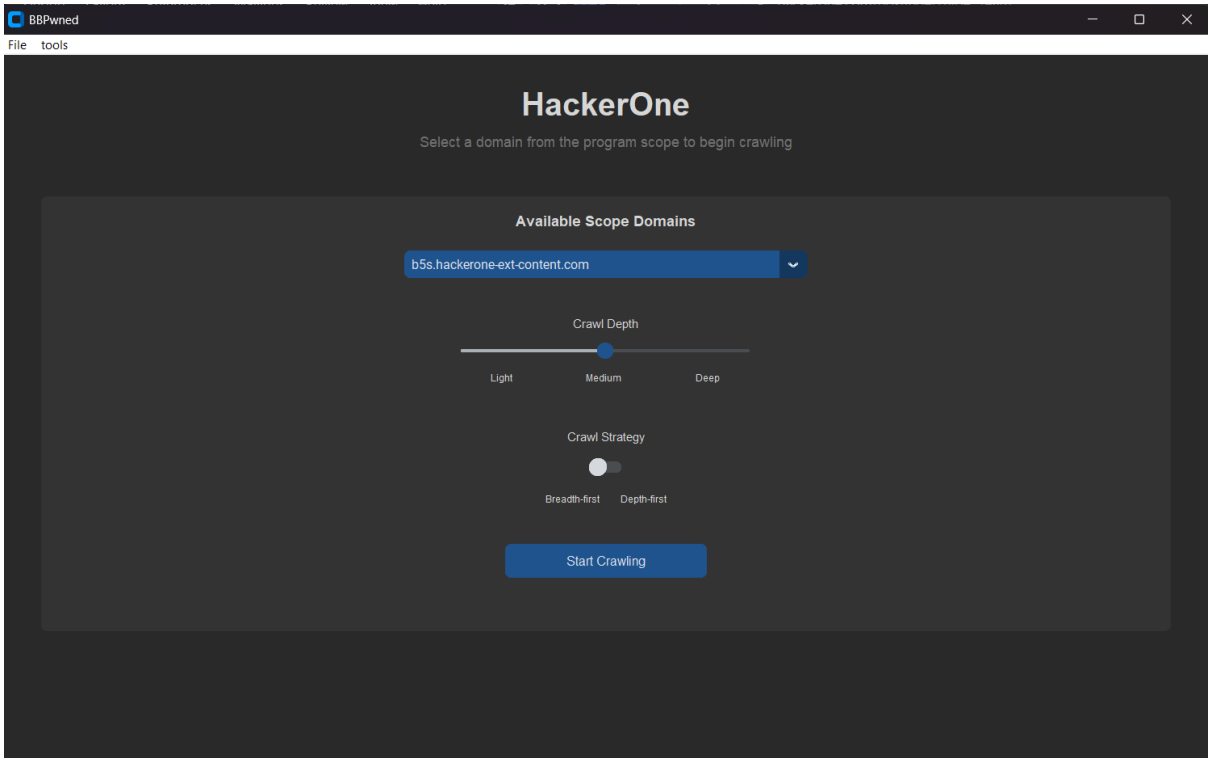
1. Navigate to the program page on HackerOne, this should show something similar to this:



2. As seen in the red circle in the picture above, the name of the program is 'HackerOne'. However, the green circle shows the directory 'security' within the URL – this is the program handle.
3. Copy this directory name (e.g. security) into the search bar on BBPwned and the program should be identified.

Web Crawling

Upon successful program selection, the toolkit will navigate to the page below:



This page is for selecting an in-scope domain and crawl parameters for the web crawl functionality. The crawl depth defines the max number of pages to crawl and the crawl strategy defines how the domain will be navigated by the crawler:

- Depth-first will prioritise going deeper into the domain.
- Breadth-first will explore all base level directories before going deeper.

Once the options are set, click ‘Start Crawling’ to begin the URL crawl. This page can be returned to at any time by clicking Tools -> Scope List within the navbar.

Directory Mapping

Once the crawl is complete, the identified directories will be displayed in a tree format as seen below:

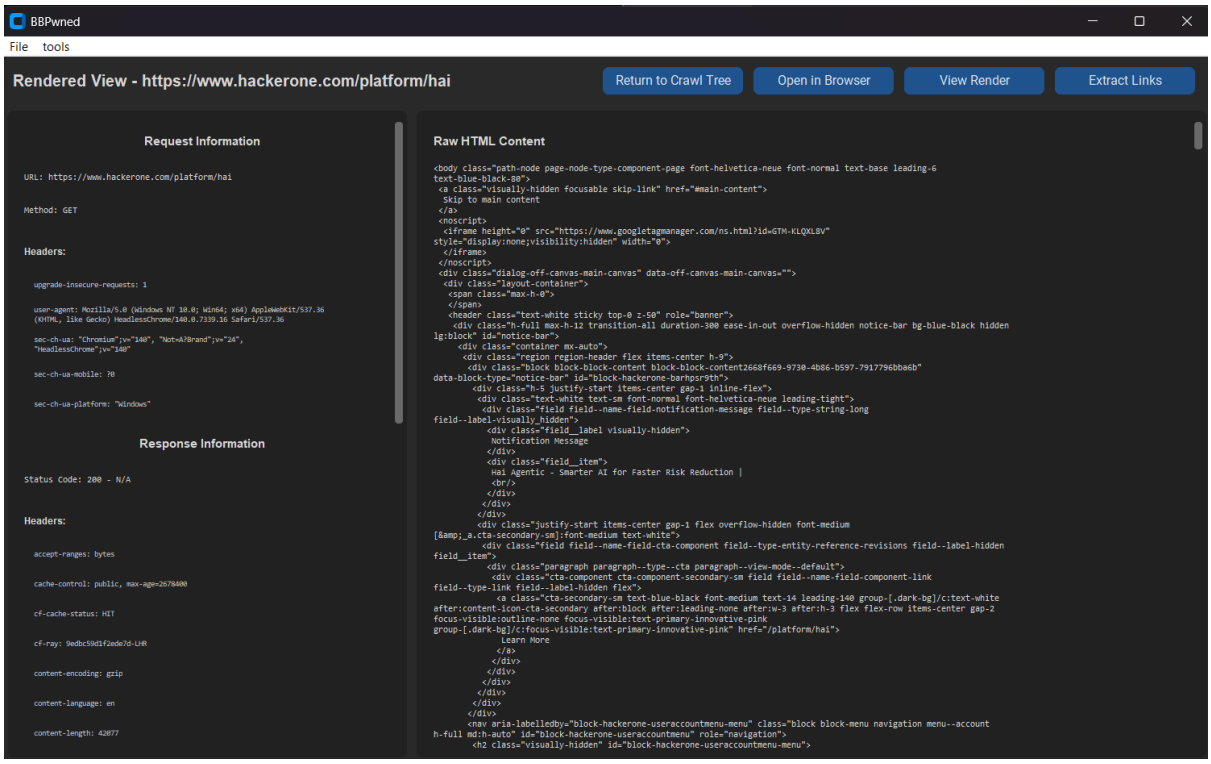
```
www.hackerone.com
└─ automotive-edition-7th-annual-hacker-powered-security-report
└─ blog
  │  └─ cost-savings-fixing-security-flaws
  │  └─ crest-and-pentesting-what-you-need-know
  │  └─ dora-what-you-need-know
  │  └─ gdpr-and-pentesting-what-you-need-know
  │  └─ general-motors-celebrates-long-term-success-hackers
  │  └─ how-crypto-and-blockchain-organizations-manage-complex-attack-surfaces-competitive-security
  │  └─ iso-27001-and-pentesting-what-you-need-know
  │  └─ nist-csf-20-and-pentesting-what-you-need-know
  │  └─ pentesting-nist-800-53-fisma-and-fedramp
  │  └─ resurrecting-shift-left-human-loop-ai
  │  └─ six-years-proactive-defense-deribits-journey-hackerone
  │  └─ soc-2-and-pentesting-what-you-need-know
  │  └─ topic
  │  │  └─ penetration-testing
└─ careers
└─ community
└─ company
  │  └─ leadership
  │  └─ newsroom
  │  └─ public-policy
```

From here, a directory can be selected for rendering, vulnerability testing and crawling further. As seen in the picture, high-value targets will be highlighted, this is based on input functionality present in the page and therefore, possible attack vectors for testing. The directory tree can be returned too by clicking Tools -> Crawl Tree within the navbar

NOTE: sometimes no directories will be identified, this is common and is due to page obfuscation strategies that protect from malicious crawlers. To fix this, render the page and extract any URLs from the result (for more information, see Page Rendering).

Page Rendering

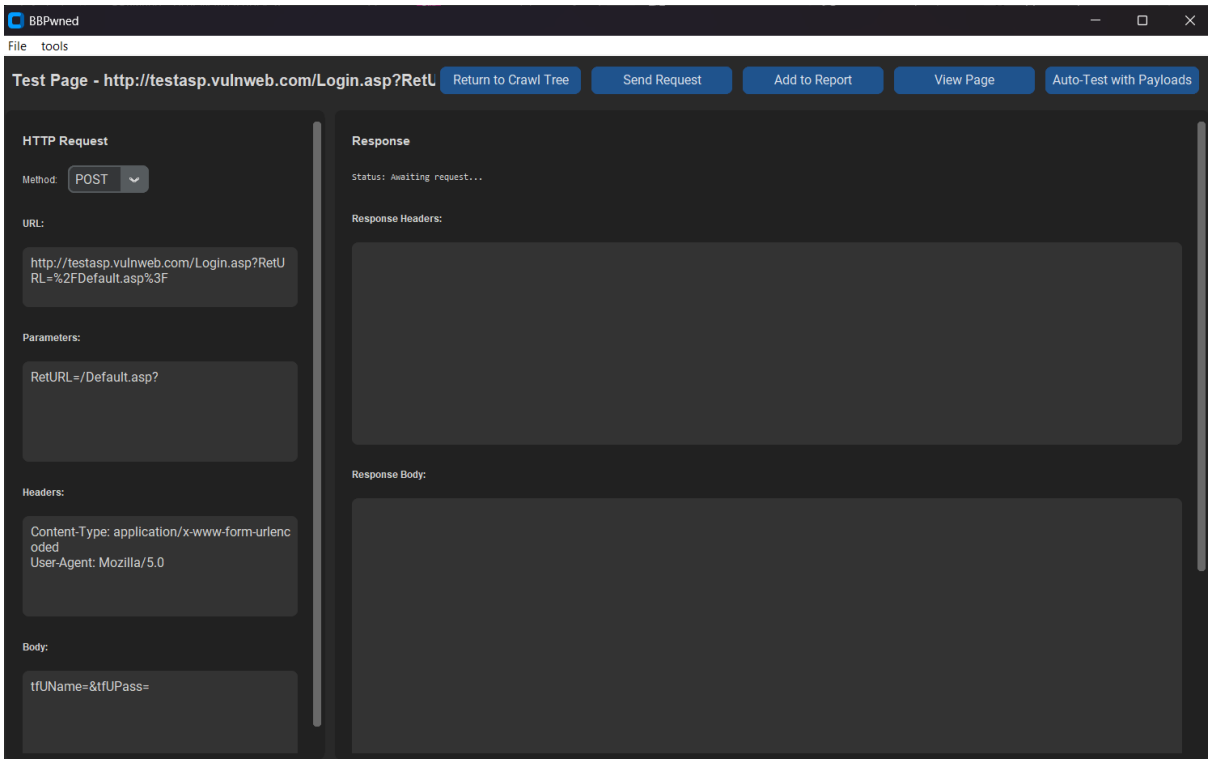
The page rendering functionality is used for thorough investigation of the page's contents, displaying the page's raw HTML, rendered output and HTTP metadata. The toolkit page is accessed by clicking 'Render Page' with a selected directory from the directory tree. Clicking this should navigate to the following page:



This can also be used to open the selected page in a browser or extract links found within the rendered content, this is particularly useful for bypassing JavaScript related obfuscation methods as previously mentioned in Directory Mapping.

Vulnerability Testing

The vulnerability testing functionality is accessed similarly to the page rendering functionality – select a directory from the directory tree and click 'Test Page'. This will open the following page:



From here, both manual and automatic testing can be conducted. The left box as seen on the picture allows for request manipulation and form/query testing – simply set the parameters appropriately and click 'Send Request'. This will result in the response being displayed within the large center box which can be analysed in text and rendered form (swapped using the View Page/View Response button) for sign of vulnerability. Any vulnerabilities identified or other noteworthy findings can be added to the final report using the 'Add To Report' button, which will open a popup window with a form for the details (for more information, see Bug Reporting).

Alternatively, the page can be automatically tested by clicking 'Auto-Test with Payloads'. This will automatically open the test results on completion and add the results to the report output (for more information, see Bug Reporting).

Bug Reporting

Any automatic results or notes added with the 'Add to Report' button from the vulnerability testing page will be added to a temporary report, this report can then be saved to the file system by clicking File -> Save Project in the navbar. This will then open the typical file save window for the installed OS. The report will save as a PDF and will follow a format similar to the following:

BBPwned - Bug Bounty Report

Program Name: Acunetix Test Site
Report Date: 2026-04-08 18:17:31

Crawled Scopes

Domain
testaspnet.vulnweb.com
testasp.vulnweb.com

Tested Pages and Results

http://testasp.vulnweb.com/Login.asp?RetURL=%2FDefault.asp%3F
■ *Forms | ? Query Params*

Vulnerabilities Found:
SQL Injection
Injection response differs significantly (Ctrl similarity: 100.0%, Test similarity: 72.4%)
LDAP Injection
LDAP injection detected via filter bypass